

User Guide: The Minimal Debugger (`minigdb`)

Mayur Kamble 2025MCS2116
Nizaul Rahman 2025MCS2099

1 Introduction

`minigdb` is a command-line debugger implemented using the `ptrace` system call on Linux. It is designed to provide essential control and inspection features necessary for low-level program analysis.

2 Setup and Execution

2.1 Compilation

The debugger is compiled using the provided `Makefile`. Navigate to the project directory in your terminal and run:

```
Terminal  
make
```

This will compile the source code (`debugger.c`) and create the executable file named `minigdb`.

2.2 Running the Debugger

To begin debugging, you must provide the path to the target executable as a command-line argument.

Usage:

```
Terminal  
./minigdb <path/to/executable>
```

Example:

```
Terminal  
./minigdb ./test_prog  
Debugger started. Debuggee PID: 726  
Debuggee stopped immediately after loading.  
  
(minigdb)
```

The program is now paused at its entry point, waiting for a command.

Note on PIE/ASLR: Due to Address Space Layout Randomization (ASLR), absolute addresses change on every run. You must find the correct address (see **Section 3.1**) before setting a breakpoint in your code.

3 Command Reference

All commands are entered at the (minigdb) prompt. Commands are case-sensitive. The integrated shell supports flexible input (e.g., ignoring leading/trailing spaces).

Command	Alias	Description
break 0xADDRESS	N/A	Sets a software breakpoint at the specified absolute memory address.
cont	N/A	Resumes normal execution of the debuggee until a breakpoint is hit or the program exits.
step	s	Executes exactly one instruction in the debuggee. If stopped at a breakpoint, it handles the single-step over the trap instruction.
info regs	N/A	Displays the current values of all general-purpose CPU registers (RIP, RSP, RAX, RBP, etc.).
status	N/A	Displays the current state of the debuggee and the signal that caused the last state change (e.g., SIGTRAP).
x [/count] 0xADDRESS	N/A	[Extension: Memory Inspection] Examines the memory at a given address. /count is optional (defaults to 64 bytes).
quit	q	Terminates the debuggee process and exits the debugger.

4 Debugging Workflow Example

This section illustrates the typical workflow, including the necessary steps to handle PIE binaries.

4.1 Step 1: Find the Absolute Breakpoint Address

Since ASLR randomizes the memory layout on every run, you must find the base address of the executable for the current PID.

1. Start Debugger & Get PID:

```
./minigdb ./test_prog
Debugger started. Debuggee PID: 726
(minigdb)
```

2. Find Base Address (New Terminal):

In a separate terminal, use the PID (e.g., 726) to find the base address.

```
cat /proc/726/maps | grep test_prog
# Example Output: 0x642f40b7e000 is the Base Address
```

3. Calculate Address:

Add the Relative Virtual Address (RVA) of your target function (e.g., 0x11aa for main from objdump) to the Base Address.

$$\text{Absolute Address} = 0x642f40b7e000 + 0x11aa = \mathbf{0x642f40b7f1aa}$$

4.2 Step 2: Set Breakpoint and Hit

Use the calculated absolute address to set the breakpoint and start execution.

```
(minigdb) break 0x642f40b7f1aa
Breakpoint set at 0x642f40b7f1aa (Original byte: 0xf3)

(minigdb) cont

*** Breakpoint hit at address 0x642f40b7f1aa ***
```

4.3 Step 3: Inspect Registers and Status

Confirm the stop reason and examine the state.

```
(minigdb) info regs
--- CPU Registers ---
RIP: 0x0000642f40b7f1aa (Instruction Pointer)
RSP: 0x00007fff7edb0a28 (Stack Pointer)
RBP: 0x0000000000000001 (Base Pointer) RAX: 0x0000642f40b7f1aa
RBX: 0x0000000000000000 RCX: 0x0000642f40b81db8
RDX: 0x00007fff7edb0b48 RDI: 0x0000000000000001
RSI: 0x00007fff7edb0b38
R8 : 0x00007571e761bf10 R12: 0x00007fff7edb0b38
R9 : 0x00007571e77bf040 R13: 0x0000642f40b7f1aa
R10: 0x00007571e77b9908 R14: 0x0000642f40b81db8
R11: 0x00007571e77d4660 R15: 0x00007571e77f3040
-----
(minigdb) status
--- Process Status ---
Status: Stopped.
Signal that caused stop: 5 (0x5)
Reason: Breakpoint/Trace event.
```

4.4 Step 4: Step and Inspect Memory (Extension Demo)

Use `step` to execute the current instruction and `x` to look at memory.

1. Step Over:

```
(minigdb) step
Single instruction executed. Stopped at 0x642f40b7f1ae.
```

2. Inspect Memory (using the current RSP):

```
(minigdb) x /32 0x7fff7edb0a28
--- Memory Dump: 0x00007fff7edb0a28 to 0x00007fff7edb0a67 ---
0x00007fff7edb0a28: 00007571e7429d90 0000000000000000 0000642f40b7f1aa
0000000100000000
0x00007fff7edb0a48: 00007fff7edb0b38 0000000000000000 eac9239436ca8250
00007fff7edb0b38
-----
```

4.5 Step 5: Quit

```
(minigdb) quit  
Terminating debuggee.
```